12-creating-custom-matchers/03/expense_tracker/spec/integration/app/ledger_spec.rb
```
expect(ledger.expenses_on('2017-06-10')).to contain_exactly(
  an_expense_identified_by(result_1.expense_id),
  an_expense_identified_by(result_2.expense_id)
)
```

It's easy to implement this new an_expense_identified_by matcher. All you have to do is write a helper method with this name that delegates to a_hash_including. To make your new matcher available to all your specs, define it in a module inside spec/spec_helper.rb. Then, configure RSpec to include this module in all your example groups:

12-creating-custom-matchers/03/expense_tracker/spec/spec_helper.rb
```
module ExpenseTrackerMatchers
  def an_expense_identified_by(id)
    a_hash_including(id: id)
  end
end

RSpec.configure do |config|
  config.include ExpenseTrackerMatchers
  # ...rest of the config...
```

There's no magic here. It's just one method delegating to another, like you do all the time in Ruby.

There is something about this method that might give you pause, though. We've defined an_expense_identified_by to match any hash containing an id key with a particular value. For example, a completely unrelated hash—such as a user record—would fool our matcher:

```
{
  id:    1,
  email: 'john.doe@example.com',
  role:  'admin'
}
```

If this matcher claims to match only expenses, you probably want to reduce the chances of a false positive. You can do so by checking for the presence of the other keys that a valid expense would contain:

12-creating-custom-matchers/04/expense_tracker/spec/spec_helper.rb
```
def an_expense_identified_by(id)
  a_hash_including(id: id).and including(:payee, :amount, :date)
end
```

Using a domain-specific name made it obvious that our match logic was too generic. This simple improvement makes the matcher more robust, and also makes it available to your entire spec suite.